

Micro proyecto 2 — Generación de código con Transformers

Curso Modelos Avanzados de Procesamiento de Lenguaje Natural

1 Introducción

En este documento se describe de manera completa el micro proyecto 2 del curso, el cual comprende el contenido de la semana 3 y 4. Además de conocer el propósito del proyecto, también se presenta la rúbrica con la que se realizará la evaluación del mismo. El documento está estructurado de la siguiente forma:

Contents

1 Introducción **1**

2 Resultados de Aprendizaje **1**

3 Descripción del micro proyecto **2**
3.1 Detalles del dataset . . . 3
3.2 Límites obligatorios sobre los datos y el entrenamiento . . . 3

4 Rúbrica de la nota del micro proyecto **3**
4.1 Implementación y entrenamiento de arquitectura GPT desde 0 . . . 4
4.2 Finetuning de los modelos pre-entrenados . . . 4
4.3 Resultados obtenidos . . . 4
4.4 Cumplimiento de requisitos de la actividad . . . 5

5 Entregables en Coursera **5**
5.1 Estructura obligatoria del Jupyter Notebook . . . 6

2 Resultados de Aprendizaje

Con esta actividad se busca que el estudiante pueda poner en práctica arquitecturas estado del arte para generación de texto y código. Tras realizar esta actividad se espera que el estudiante esté en capacidad de:

1. Estructurar un corpus (en este caso, de código fuente) de forma que pueda ser usado para entrenar un modelo de lenguaje.
2. Utilizar la arquitectura de Transformers para implementar modelos de lenguaje desde cero.
3. Utilizar modelos pre-entrenados como punto de inicio para el entrenamiento de modelos especializados.

4. Hacer finetuning de un modelo pre-entrenado para mejorar el desempeño en una tarea específica.

3 Descripción del micro proyecto

El objetivo del micro proyecto es entrenar un modelo de lenguaje basado en Transformers capaz de generar código fuente en Python. La generación de código es una de las aplicaciones más relevantes de los modelos de lenguaje actuales (asistentes de programación tipo Copilot), y permite evaluar de forma cualitativa muy clara la calidad de las generaciones (indentación, sintaxis, estructura de funciones, etc.).

Usted debe comparar el desempeño de cinco configuraciones de modelo, organizadas en torno a dos enfoques:

1. Modelo desde cero: un modelo Transformer *decoder-only* (tipo GPT) entrenado desde cero en PyTorch. Puede hacer la implementación completamente desde cero (mecanismo de atención, *multi-head attention* y *embeddings* posicionales, como en el tutorial de la semana 4) o utilizar como base la clase `Transformer` de la librería.

2. Modelos pre-entrenados (familia SmolLM2): debe trabajar con los modelos `HuggingFaceTB/SmolLM2-135M` y `HuggingFaceTB/SmolLM2-360M`, ambos **obligatorios**; no se permite usar otros modelos pre-entrenados. Para cada uno de los dos tamaños debe evaluar dos versiones: la versión pre-entrenada tal cual (sin finetuning, como *baseline*) y la versión resultante de su finetuning completo sobre el corpus de código. Al tratarse de modelos de propósito general (no especializados en código), el finetuning permite especializarlos en la tarea, y contrastar cada *baseline* contra su finetuning hace que la mejora obtenida sea clara y medible.

En total se comparan cinco configuraciones: el modelo desde cero, y para cada tamaño (135M y 360M) la versión pre-entrenada y la versión con finetuning completo.

Para que el desempeño sea comparable a través de la perplejidad, el tamaño del vocabulario (y el tokenizador) debe ser el mismo para todas las configuraciones. Esto se cumple de forma natural en la familia SmolLM2: tanto el modelo de 135M como el de 360M comparten exactamente el mismo tokenizador por subpalabras (BPE), con un vocabulario de 49.152 tokens. El modelo entrenado desde cero debe utilizar ese mismo tokenizador.

Compare el desempeño de las cinco configuraciones sobre el conjunto de prueba, tanto mediante la perplejidad como mediante la validez sintáctica de las generaciones (ver la sección de resultados), y realice por lo menos 3 ejemplos de generación de código con cada uno de los modelos entrenados por usted (modelo desde cero, 135M finetuned y 360M finetuned), a modo cualitativo (la función de generación debe implementarse desde cero para todas las configuraciones, sin usar funciones como `model.generate()`; ver los requisitos de la actividad).

Nota sobre el finetuning. En este punto del curso aún no se han visto técnicas de *finetuning* eficiente (como LoRA/QLoRA), por lo que se espera que realice un finetuning completo de ambos modelos pre-entrenados (actualizando todos sus parámetros), tal como se hace con la clase `AutoModelForCausalLM` de Hugging Face. Por su tamaño, los modelos SmolLM2 de 135M y 360M pueden entrenarse por completo con recursos de cómputo modestos.

Recomendaciones para evitar el cuello de botella de memoria. No es necesario que los dos modelos estén en memoria al mismo tiempo: entrene y finetunee los modelos de a uno, guardando el *checkpoint* de cada modelo y liberando la GPU antes de cargar el siguiente (por ejemplo, con `del model`, `del optimizer` y `torch.cuda.empty_cache()`). De esta forma el caso más exigente es el finetuning completo del modelo de 360M por sí solo. Para que quepa en una GPU modesta, dicho finetuning **requiere** usar: un tamaño de *batch* pequeño (1–4) con acumulación de gradiente, precisión mixta (`bf16`/`fp16`) y gradient checkpointing; opcionalmente, un optimizador en 8 bits (por ejemplo, mediante `bitsandbytes`) para reducir la memoria de los estados del optimizador. Las versiones pre-entrenadas usadas como *baseline* solo se evalúan (inferencia), por lo que su consumo de memoria es mucho menor.

El modelo pre-entrenado y su tokenizador pueden cargarse de la siguiente manera mediante la librería `transformers` de Hugging Face:

```
from transformers import AutoTokenizer, AutoModelForCausalLM

# Ambos modelos comparten el mismo tokenizador (vocabulario de 49152)
for model_name in ["HuggingFaceTB/SmolLM2-135M", "HuggingFaceTB/SmolLM2-360M"]:
    tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
    model = AutoModelForCausalLM.from_pretrained(model_name)
    print(model_name)
    print(" Tamano del vocabulario:", tokenizer.vocab_size) # 49152
    print(" Parametros:", sum(p.numel() for p in model.parameters()))
```

3.1 Detalles del dataset

El dataset se compone de un corpus abierto de código fuente en Python. Debe utilizar **obligatoriamente** el dataset `codeparrot/codeparrot-clean` (código Python recopilado de repositorios públicos de GitHub), disponible en Hugging Face. No está permitido cambiar de dataset ni usar datos de otras fuentes. Dado que el corpus completo es muy grande, debe trabajar con un subconjunto sujeto a los límites obligatorios descritos en la Sección 3.2 (filtro de longitud, número máximo de ejemplos y longitud de contexto). A partir de este corpus, usted debe definir las particiones de entrenamiento, validación y prueba según el **split obligatorio**; las particiones deben poder generarse dentro del código del *notebook* que va a entregar.

```
from datasets import load_dataset

# Se carga un subconjunto del corpus de codigo Python (modo streaming para
# no descargar todo el dataset). Ajuste el numero de ejemplos a sus recursos.
stream = load_dataset("codeparrot/codeparrot-clean", split="train", streaming=True)

n_examples = 8000
min_chars, max_chars = 1000, 2000 # filtro de longitud obligatorio
corpus = []
for example in stream:
    code = example["content"] # 'content' contiene el codigo del archivo
    if not (min_chars <= len(code) <= max_chars):
        continue # se descartan ejemplos fuera del rango
    corpus.append(code)
    if len(corpus) >= n_examples:
        break

print("Ejemplos cargados:", len(corpus))
print(corpus[0][:500])
```

Recuerde tokenizar el corpus con el mismo tokenizador del modelo pre-entrenado y concatenar/segmentar el código en secuencias del tamaño de contexto que defina para el entrenamiento.

3.2 Límites obligatorios sobre los datos y el entrenamiento

Para el entrenamiento de ambos modelos usted debe respetar los siguientes límites, todos obligatorios:

- **Filtro de longitud:** conserve únicamente los ejemplos del corpus cuyo contenido (`content`) tenga entre **1.000 y 2.000 caracteres**. Esto descarta los archivos muy largos que no caben en el contexto y deja ejemplos manejables que pueden regenerarse y verificarse por completo.
- **Máximo 8.000 ejemplos** del corpus tras aplicar el filtro de longitud.
- **Longitud de contexto fija de 256 tokens** para ambos modelos. Este valor debe ser el mismo en el modelo entrenado desde cero y en el finetuning, de modo que la pérdida sea comparable entre ambos.

Esos 8.000 ejemplos deben repartirse en una división **80 / 10 / 10**, generada dentro del *notebook*:

- **6.400 ejemplos para entrenamiento.**
- **800 ejemplos para validación.**
- **800 ejemplos para prueba.**

Estos límites mantienen el proyecto resoluble con recursos de cómputo modestos y aseguran tiempos de entrenamiento razonables.

4 Rúbrica de la nota del micro proyecto

La nota del micro proyecto se compone de la forma descrita en la siguiente tabla:

Item	Porcentaje de la nota
Implementación y entrenamiento de arquitectura GPT desde 0	10%
Finetuning de los modelos pre-entrenados	50%
Resultados obtenidos	40%

Table 1: Composición de la nota del micro proyecto

4.1 Implementación y entrenamiento de arquitectura GPT desde 0

El estudiante debe implementar una arquitectura GPT (*decoder-only*) para el entrenamiento del modelo en el corpus de código dado. Debe utilizar el mismo tamaño de vocabulario (y tokenizador) de los modelos pre-entrenados (el tokenizador de SmolLM2, con vocabulario de 49.152), de modo que la perplejidad sea comparable entre todas las configuraciones. Se puede guiar del tutorial de la semana 4.
Hiperparámetros recomendados. Como punto de partida (puede ajustarlos según lo que observe en la pérdida de validación): tasa de aprendizaje $\approx 3 \times 10^{-4}$ con el optimizador `AdamW`, entrenado durante ≈ 3 épocas sobre el conjunto de entrenamiento.

4.2 Finetuning de los modelos pre-entrenados

El estudiante debe cargar los modelos de lenguaje pre-entrenados `HuggingFaceTB/SmolLM2-135M` y `HuggingFaceTB/SmolLM2-360M` junto con sus pesos para luego realizar el finetuning completo de cada uno sobre el corpus de código Python. No se permite, en este micro proyecto, sustituir el finetuning completo por técnicas de finetuning eficiente (LoRA/QLoRA), pues estas se abordan más adelante en el curso. Recuerde que este finetuning completo consume bastante memoria; para reducir ese consumo apóyese en precisión mixta, acumulación de gradiente y gradient checkpointing, tal como se explica en el documento anexo `precision_mixta/precision_mixta_gradient_accumulation_checkpointing.pdf`, que describe en detalle cada técnica y su implementación en PyTorch/Hugging Face.
Hiperparámetros recomendados. Como punto de partida (puede ajustarlos según lo que observe en la pérdida de validación):

- SmolLM2-135M: tasa de aprendizaje $\approx 5 \times 10^{-5}$ (`AdamW`); dado el tamaño del corpus, 1 época completa suele ser suficiente.
- SmolLM2-360M: tasa de aprendizaje $\approx 3 \times 10^{-5}$ (`AdamW`), también con 1 época completa.

Estos valores son un punto de partida razonable, no un requisito estricto: monitoree la pérdida de validación durante el entrenamiento y ajuste la tasa de aprendizaje o el número de épocas si el modelo aún no converge o si empieza a sobreajustar. Combine estos hiperparámetros con las técnicas de precisión mixta, acumulación de gradiente y *gradient checkpointing* del documento anexo antes mencionado para que el finetuning quepa en una GPU modesta.

4.3 Resultados obtenidos

El estudiante debe evaluar la capacidad de generalización de los modelos sobre el conjunto de prueba (los 800 ejemplos del conjunto de prueba definidos en el split obligatorio). El conjunto de validación se usa para el monitoreo durante el entrenamiento y el ajuste de hiperparámetros, mientras que el conjunto de prueba se reserva exclusivamente para el reporte final. Se evaluará el grado en el que los modelos finetuneados demuestran tener la capacidad de ajustarse a los datos, y las acciones realizadas para evitar el sobre ajuste.
Como métricas de evaluación obligatorias, reporte sobre el conjunto de prueba, para cada una de las cinco configuraciones (modelo desde cero; 135M pre-entrenado y finetuned; 360M pre-entrenado y finetuned). La perplejidad es la métrica cuantitativa principal de la comparación; la validez sintáctica es una métrica secundaria de apoyo (es más ruidosa; ver la nota tras la lista):

- **Perplejidad (métrica principal)** sobre el conjunto de prueba, calculada a nivel de token. En cada posición, el modelo produce una distribución sobre el vocabulario condicionada en los tokens anteriores; se toma la probabilidad $p(x_i \mid x_{<i})$ que asigna al token verdadero x_i. La perplejidad es la exponencial del promedio de la log-verosimilitud negativa por token:

$$\mathrm{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log p(x_i \mid x_{<i}) \right),$$

donde N es el número total de tokens evaluados (equivale a la exponencial de la pérdida promedio por token). Acumule $-\log p(x_i \mid x_{<i})$ sobre *todos* los tokens del conjunto de prueba y divida por N antes de exponenciar; no promedie las perplejidades por lote, y si emplea relleno (*padding*) excluya esas posiciones del conteo.

- **Validez sintáctica de las generaciones (métrica secundaria).** Sobre una muestra de **200 ejemplos** del conjunto de prueba (con semilla fija para reproducibilidad), tome un prefijo del código como *prompt*, genere una continuación y verifique con `ast.parse` si el código resultante es sintácticamente válido (sin ejecutarlo). Antes de verificar, descarte la última línea si quedó truncada por el límite de tokens. La métrica es el porcentaje de generaciones válidas.

Interpretación. La validez sintáctica es una métrica secundaria y ruidosa (se calcula sobre pocos ejemplos), así que no sobreinterprete diferencias pequeñas: el finetuning puede reducir la perplejidad sin que la validez cambie de forma apreciable. Apóyese en la perplejidad para las conclusiones cuantitativas. El estudiante debe hacer un análisis comparativo del desempeño de las cinco configuraciones a partir de estas métricas y de las curvas de la función de costo durante el entrenamiento. El análisis debe contrastar explícitamente cada modelo pre-entrenado (*baseline*) contra su versión con finetuning, y los dos tamaños (135M frente a 360M) entre sí. Adicionalmente, a modo cualitativo, debe incluir al menos **3 ejemplos de generación de código** por cada uno de los modelos entrenados por usted: el modelo desde cero, el 135M finetuned y el 360M finetuned (recuerde que la función de generación debe implementarse desde cero para todas las configuraciones, sin usar `model.generate()`; ver los requisitos de la actividad).

4.4 Cumplimiento de requisitos de la actividad

Recuerde que su entrega no tendrá validez si no cumple los requisitos que se mencionan a continuación. Por lo tanto, obtendrá una calificación de 0 en la actividad correspondiente al micro proyecto.

Los requisitos son los siguientes:

- El dataset obligatorio es `codeparrot/codeparrot-clean` (link). **No se permite cambiar de dataset ni usar datos de otras fuentes** (de ningún tipo) para el entrenamiento de los modelos.
- Debe implementar y entrenar los modelos haciendo uso de PyTorch, y de las librerías que se le haya indicado únicamente (por ejemplo, `transformers` y `datasets` de Hugging Face).
- Los modelos pre-entrenados obligatorios son `HuggingFaceTB/SmolLM2-135M` (link) y `HuggingFaceTB/SmolLM2-360M` (link). No se permite usar otros modelos pre-entrenados.
- El finetuning de ambos modelos pre-entrenados debe ser un finetuning completo (no se permite LoRA/QLoRA en este micro proyecto).
- La **función de generación debe implementarse desde cero para todas las configuraciones**, incluidos los modelos pre-entrenados y finetuneados de Hugging Face. Esto implica escribir usted mismo el bucle autorregresivo: obtener los *logits* del modelo en cada paso, transformarlos en una distribución sobre el vocabulario, muestrear (o seleccionar) el siguiente token y concatenarlo al contexto para el siguiente paso. **No se permite** usar funciones de generación de alto nivel de las librerías (por ejemplo, `model.generate()` de Hugging Face).
- El código enviado debe poder ser ejecutado por el equipo evaluador, y el código debe ser completamente de su autoría.
- Debe crear y entregar los *checkpoints* de los tres modelos entrenados por usted (modelo desde cero, 135M finetuned y 360M finetuned), guardados mediante `torch.save`; para los dos modelos finetuneados también se acepta `save_pretrained` de Hugging Face. Deben incluirse en el archivo zip de entregables (ver la sección de entregables). Las versiones pre-entrenadas usadas como *baseline* no requieren entregarse, pues se descargan directamente de Hugging Face.

5 Entregables en Coursera

En la semana 4 del curso se habilitará una actividad en Coursera. Dado el tamaño de los *checkpoints*, en esa actividad usted no sube archivos: debe enviar **un enlace de Google Drive** a un archivo comprimido (zip) que contenga los siguientes entregables:

1. Jupyter Notebook que se utilizó para realizar la implementación y el entrenamiento de los modelos, junto con los análisis de los resultados obtenidos.

2. Los *checkpoints* de los tres modelos entrenados por usted (modelo desde cero, finetuning de 135M y finetuning de 360M): un archivo por modelo si usó `torch.save`, o una carpeta por modelo si usó `save_pretrained`.

El archivo zip debe estar compartido en Google Drive con el permiso *Cualquier persona que tenga el vínculo* (lector). Verifique, desde una ventana de incógnito, que el enlace permite descargar el archivo sin pedir acceso, y no modifique ni elimine el archivo hasta que finalice el proceso de calificación. Un enlace que no se pueda abrir equivale a no entregar.

5.1 Estructura obligatoria del Jupyter Notebook

El Jupyter Notebook debe organizarse *obligatoriamente* en las siguientes secciones, en este mismo orden y con estos mismos títulos. El notebook debe ser autocontenido y ejecutable de principio a fin (*Run All*). Entre paréntesis se indica el ítem de la rúbrica al que contribuye cada sección:

1. **Portada e introducción.** Nombre del estudiante, resumen del proyecto, descripción de las cinco configuraciones que se compararán (modelo desde cero; SmolLM2-135M y SmolLM2-360M, cada uno pre-entrenado y con finetuning).

2. **Librerías.** Importación de todas las dependencias utilizadas.

3. **Carga del dataset y exploración.** Carga en modo *streaming* de `codeparrot/codeparrot-clean` y exploración del corpus (por ejemplo, distribución de longitudes y ejemplos de código).

4. **Preprocesamiento.** Filtro de longitud (1.000–2.000 caracteres), tokenización con el tokenizador de SmolLM2 (compartido por el 135M y el 360M) y segmentación del corpus en secuencias de 256 tokens.

5. **Split train / validación / prueba.** Aplicación del split obligatorio (6.400 / 800 / 800).

6. **Modelo desde cero.** Implementación en PyTorch de la arquitectura GPT (*decoder-only*) y su entrenamiento sobre el corpus. *(Implementación y entrenamiento de arquitectura GPT desde 0)*

7. **Finetuning de los modelos pre-entrenados.** Carga de `SmolLM2-135M` y `SmolLM2-360M` con sus pesos y finetuning completo de cada uno sobre el corpus de código. *(Finetuning de los modelos pre-entrenados)*

8. **Evaluación y comparación.** Perplejidad y porcentaje de validez sintáctica sobre el conjunto de prueba para las cinco configuraciones (incluyendo los *baselines* pre-entrenados sin finetuning), con su análisis comparativo. *(Resultados obtenidos)*

9. **Generación de ejemplos.** Función de generación implementada desde cero (bucle autorregresivo a partir de los *logits*, sin `model.generate()`) y al menos 3 ejemplos de código generado por cada modelo entrenado por usted (desde cero, 135M finetuned y 360M finetuned). *(Resultados obtenidos)*

10. **Conclusiones.** Discusión de los resultados, del sobre ajuste y de las limitaciones de los modelos.